Lab Exercise 15- Creating ConfigMaps in Kubernetes

In this lab exercise, you will learn how to create ConfigMaps in Kubernetes using different methods: from literal values, from a file, from an environment file, and from multiple files. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a ConfigMap from Literal Values

Create a ConfigMap named example-configmap with literal values:

```
kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
```

Verify the ConfigMap:

```
kubectl get configmap example-configmap -o yaml
```

You should see the example-configmap with the keys and values specified.

```
MINGW64:/c/Users/namit

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
error: failed to create configmap: namespaces "my-namespace" not found

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl config set-context --current --namespace=default
Context "docker-desktop" modified.

namit@ThinkPadE15 MINGW64 ~ (main)
$  kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
configmap/example-configmap created

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl get configmap example-configmap -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-16T11:14:03Z"
  name: example-configmap
  namespace: default
  resourceVersion: "46519"
  uid: 27ed6dd2-d242-4ed2-aced-4f593753f590

namit@ThinkPadE15 MINGW64 ~ (main)
$
```

Step 3: Create a ConfigMap from a File

Create a file named **config.txt** with the following content:

```
key1=value1
key2=value2
```

Create a ConfigMap named file-configmap from the file:

```
kubectl create configmap file-configmap --from-file=config.txt
```

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ notepad config.txt

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl create configmap file-configmap --from-file=config.txt
configmap/file-configmap created

namit@ThinkPadE15 MINGW64 ~ (main)
$ |
```

Verify the ConfigMap:

```
kubectl get configmap file-configmap -o yaml
```

You should see the file-configmap with the content of config.txt.

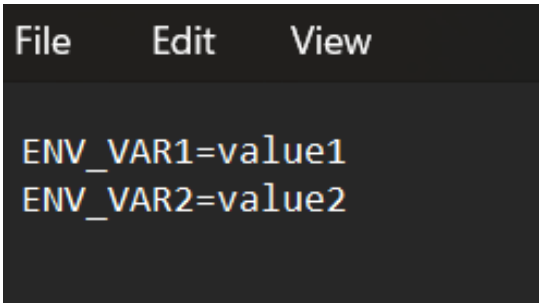
```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl get configmap file-configmap -o yaml
apiVersion: v1
data:
  config.txt: "key1=value1\r\nkey2=value2\r\n"
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-16T11:17:47Z"
  name: file-configmap
  namespace: default
  resourceVersion: "46818"
  uid: 1db33427-062d-43f7-a511-b92a9e2e164a

namit@ThinkPadE15 MINGW64 ~ (main)
$ |
```

Step 4: Create a ConfigMap from an Environment File

Create an environment file named **env-config.env** with the following content:

```
ENV_VAR1=value1
ENV_VAR2=value2
```



Create a ConfigMap named env-configmap from the environment file:

```
kubectl create configmap env-configmap --from-env-file=env-config.env
```

Verify the ConfigMap:

```
kubectl get configmap env-configmap -o yaml
```

You should see the env-configmap with the environment variables specified in the file.

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl create configmap env-configmap --from-env-file=env-config.env
configmap/env-configmap created

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl get configmap env-configmap -o yaml
apiVersion: v1
data:
  ENV_VAR1: value1
  ENV_VAR2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-16T11:20:14Z"
  name: env-configmap
  namespace: default
  resourceVersion: "47016"
  uid: 626341af-358e-4552-94a6-805fa37cb72e

namit@ThinkPadE15 MINGW64 ~ (main)
$ |
```

Step 5: Create a ConfigMap from Multiple Files

Create multiple files:

config1.txt with content:

```
key1=value1
```

config2.txt with content:

```
key2=value2
```

Create a ConfigMap named multi-file-configmap from multiple files:

```
kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
```

Verify the ConfigMap:

```
kubectl get configmap multi-file-configmap -o yaml
```

You should see the multi-file-configmap with the content of both config1.txt and config2.txt.

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
configmap/multi-file-configmap created

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl get configmap multi-file-configmap -o yaml
apiVersion: v1
data:
  config1.txt: key1=value1
  config2.txt: key2=value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-16T11:22:05Z"
  name: multi-file-configmap
  namespace: default
  resourceVersion: "47165"
  uid: 212e6f03-5813-40fb-a862-784148e15ac0

namit@ThinkPadE15 MINGW64 ~ (main)
$
```

Step 6: Use ConfigMaps in a Pod

Create a file named **pod-configmap.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

In this manifest:

- The Pod reads environment variables from the env-configmap.
- The Pod mounts the file-configmap at /etc/config.

Apply the manifest to create the Pod:

```
kubectl apply -f pod-configmap.yaml
```

Check the status of the Pod:

```
kubectl get pods
```

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl apply -f pod-configmap.yaml
pod/configmap-pod created

namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl get pods
NAME            READY   STATUS              RESTARTS   AGE
configmap-pod   0/1     ContainerCreating   0          19s

namit@ThinkPadE15 MINGW64 ~ (main)
$ |
```

View the Pod logs to see the environment variables and file content:

```
kubectl exec -it configmap-pod -- printenv
```

You should see the environment variables and the content of the file-configmap.

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl exec -it configmap-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-pod
TERM=xterm
ENV_VAR2=value2
ENV_VAR1=value1
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root

namit@ThinkPadE15 MINGW64 ~ (main)
$ |
```

Step 7: Clean Up

After completing the exercise, clean up the resources created:

```
kubectl delete pod configmap-pod
kubectl delete configmap example-configmap
kubectl delete configmap file-configmap
kubectl delete configmap env-configmap
kubectl delete configmap multi-file-configmap
rm config.txt env-config.env config1.txt config2.txt
```

```
namit@ThinkPadE15 MINGW64 ~ (main)
$ kubectl delete pod configmap-pod
kubectl delete configmap example-configmap
kubectl delete configmap file-configmap
kubectl delete configmap env-configmap
kubectl delete configmap multi-file-configmap
rm config.txt env-config.env config1.txt config2.txt
pod "configmap-pod" deleted
configmap "example-configmap" deleted
configmap "file-configmap" deleted
configmap "env-configmap" deleted
configmap "multi-file-configmap" deleted

namit@ThinkPadE15 MINGW64 ~ (main)
```